

8. Firewall Configuration Basics: Configure firewall rules to control incoming and outgoing network traffic .

1. Aim

To understand the working of a firewall and implement a firewall rule simulation in Python to control incoming and outgoing network traffic based on source IP address, destination IP address, protocol, port number, direction, and security action.

2. Objectives

After completing this practical, students will be able to:

1. Understand the concept of a firewall.
2. Differentiate between inbound and outbound traffic.
3. Understand firewall rules and policies.
4. Identify the role of IP addresses and port numbers in traffic filtering.
5. Understand TCP and UDP traffic filtering.
6. Implement allow and block firewall rules.
7. Apply a default-deny security policy.
8. Test network traffic against firewall rules.
9. Generate firewall activity logs.
10. Calculate allowed and blocked traffic statistics.
11. Understand how actual firewall configuration differs from firewall simulation.
12. Execute the complete practical in Google Colab.

3. Introduction

A firewall is a network security mechanism used to monitor, filter, and control network traffic between different systems or networks.

A firewall examines network packets and compares their characteristics with a set of predefined rules. Based on these rules, the firewall decides whether the traffic should be:

- Allowed
- Blocked
- Rejected
- Logged

A firewall can control traffic based on several parameters:

Source IP Address

Destination IP Address

Protocol

Source Port

Destination Port

Traffic Direction

Application

Network Interface

Action

For example, an organization may want to allow HTTPS traffic while blocking Telnet:

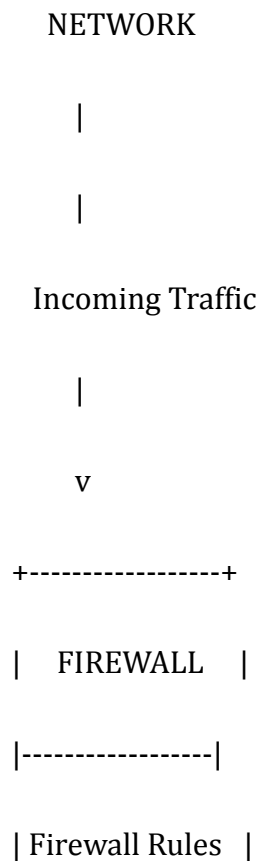
TCP Port 443 → ALLOW

TCP Port 23 → BLOCK

The firewall therefore acts as a traffic control mechanism.

4. Basic Firewall Concept

The basic operation of a firewall is:



+-----+

| |

| |

ALLOW BLOCK

| |

v v

Protected Dropped

Network Traffic

The firewall receives network traffic and checks whether the traffic matches an existing rule.

For example:

Traffic:

Protocol = TCP

Port = 443

Firewall Rule:

TCP Port 443 = ALLOW

Decision:

ALLOW

Another example:

Traffic:

Protocol = TCP

Port = 23

Firewall Rule:

TCP Port 23 = BLOCK

Decision:

BLOCK

5. Incoming Traffic

Incoming traffic, also called inbound traffic, is traffic coming into a computer, server, or network.

Example:

Internet

|

v

Firewall

|

v

Computer

Examples include:

- HTTP requests
- HTTPS requests
- SSH connections
- Remote Desktop connections
- File-sharing requests
- Database connection attempts
- Ping/ICMP traffic

For example:

Source IP = 192.168.1.50

Destination IP = Local Computer

Protocol = TCP

Port = 443

Direction = INBOUND

If the firewall contains:

TCP 443 → ALLOW

the traffic is allowed.

6. Outgoing Traffic

Outgoing traffic, also called outbound traffic, is traffic leaving the local computer or network.

Example:

Computer

|

v

Firewall

|

v

Internet

Examples include:

- Web browsing
- DNS requests
- API requests
- Software updates
- Email
- File uploads
- Cloud services

Example:

Source = Local Computer

Destination = 8.8.8.8

Protocol = UDP

Port = 53

Direction = OUTBOUND

A firewall can determine whether this traffic should be allowed.

7. Firewall Rules

A firewall rule defines how the firewall should handle traffic that matches particular conditions.

A rule can contain:

Parameter	Description
Rule ID	Unique identification number
Rule Name	Name of the rule
Direction	Inbound or outbound
Source	Source IP address
Destination	Destination IP address
Protocol	TCP, UDP, ICMP, etc.
Port	Network service port
Action	Allow or Block

Example:

Rule Name : Allow HTTPS

Direction : INBOUND

Source : ANY

Destination : LOCAL

Protocol : TCP

Port : 443

Action : ALLOW

Meaning:

Allow incoming TCP traffic destined for port 443.

8. Common Network Ports

Port	Protocol	Common Service
20	TCP	FTP Data
21	TCP	FTP Control
22	TCP	SSH
23	TCP	Telnet
25	TCP	SMTP
53	UDP/TCP	DNS
67/68	UDP	DHCP
80	TCP	HTTP

110	TCP	POP3
143	TCP	IMAP
443	TCP	HTTPS
3389	TCP	Remote Desktop

These are conventional service ports; actual applications can sometimes use different ports.

9. TCP

Transmission Control Protocol (TCP) is a connection-oriented transport protocol.

It provides:

- Reliable delivery
- Ordered data transmission
- Error detection
- Connection management

Common TCP-based services include:

HTTP

HTTPS

SSH

FTP

SMTP

Example:

TCP + Port 443

is commonly associated with HTTPS.

10. UDP

User Datagram Protocol (UDP) is a connectionless transport protocol.

It generally provides lower overhead than TCP but does not provide TCP's connection-oriented reliability mechanisms.

Common UDP-based services include:

DNS

DHCP

Streaming

Voice communication

Online gaming

Example:

UDP + Port 53

is commonly used for DNS.

11. Source IP Address

The source IP address identifies where the network traffic originates.

Example:

Source IP:

192.168.1.100

A firewall can use source IP addresses to restrict access.

Example:

192.168.1.100 → Port 22 → ALLOW

ANY OTHER IP → Port 22 → BLOCK

This provides more restrictive access control.

12. Destination IP Address

The destination IP address identifies the system receiving the traffic.

Example:

Source:

192.168.1.100

Destination:

192.168.1.10

Firewall rules can use both source and destination addresses.

13. Allow Rule

An ALLOW rule permits traffic that matches the configured conditions.

Example:

Protocol = TCP

Port = 443

Action = ALLOW

This allows TCP traffic through port 443 when the rule matches.

14. Block Rule

A BLOCK rule prevents matching traffic.

Example:

Protocol = TCP

Port = 23

Action = BLOCK

This blocks TCP traffic to port 23.

Telnet commonly uses port 23 and is generally avoided in favor of more secure remote-access mechanisms such as SSH.

15. Default-Deny Policy

The practical uses a default-deny policy.

This means:

If traffic does not match an explicit allow rule, it is blocked.

Example:

TCP 80 → ALLOW

TCP 443 → ALLOW

UDP 53 → ALLOW

TCP 23 → BLOCK

TCP 22 → No matching rule → BLOCK

Default-deny is commonly associated with the principle of least privilege.

16. Principle of Least Privilege

The principle of least privilege means providing only the access that is required.

For network security:

Required traffic → ALLOW

Unnecessary traffic → BLOCK

Instead of:

Allow Everything

the security administrator should define only the necessary services.

17. Firewall Processing Model

The Python simulation follows this model:

Network Packet

v

Is Firewall Enabled?

/ \

NO YES

| |

ALLOW v

Check Firewall Rules

|

v

Match Rule Found?

/ \

YES NO

| |

v v

Rule Action DEFAULT DENY

| |

ALLOW/BLOCK BLOCK

18. Complete Google Colab Code

The following is the complete executable program. It does not require external libraries and can be pasted into one Google Colab cell and executed directly.

```
# =====
```

```
# PRACTICAL 8
```

```
# FIREWALL CONFIGURATION BASICS
```

```
#
```

```
# Configure Firewall Rules to Control Incoming and
```

```
# Outgoing Network Traffic
```

```
#
```

```
# Platform : Google Colab
```

```
# Language : Python
```

```
# Type : Firewall Rule Simulation
```

```
# =====
```

```
# =====
```

```
# 1. IMPORT REQUIRED MODULE
```

```
# =====
```

```
from datetime import datetime
```

```
# =====
```

```
# 2. FIREWALL CONFIGURATION
```

```
# =====
```

```
firewall_enabled = True
```

```
# Default security policy
```

```
# If no rule matches, traffic will be BLOCKED.
```

```
default_policy = "BLOCK"
```

```
# =====
```

```
# 3. FIREWALL RULE DATABASE
```

```
# =====
```

```
firewall_rules = [
```

```
# -----
```

```
# INBOUND RULES
```

```
# -----
```

```
{
```

```
    "id": 1,
```

```
    "name": "Allow HTTP",
```

```
    "direction": "INBOUND",
```

```
    "source": "ANY",
```

```
"destination": "LOCAL",  
  
"protocol": "TCP",  
  
"port": 80,  
  
"action": "ALLOW"  
},
```

```
{  
  
  "id": 2,  
  
  "name": "Allow HTTPS",  
  
  "direction": "INBOUND",  
  
  "source": "ANY",  
  
  "destination": "LOCAL",  
  
  "protocol": "TCP",  
  
  "port": 443,  
  
  "action": "ALLOW"  
},
```

```
{  
  
  "id": 3,  
  
  "name": "Allow SSH from Admin",  
  
  "direction": "INBOUND",  
  
  "source": "192.168.1.100",
```

```
"destination": "LOCAL",  
  
"protocol": "TCP",  
  
"port": 22,  
  
"action": "ALLOW"  
},
```

```
{  
  
  "id": 4,  
  
  "name": "Block Telnet",  
  
  "direction": "INBOUND",  
  
  "source": "ANY",  
  
  "destination": "LOCAL",  
  
  "protocol": "TCP",  
  
  "port": 23,  
  
  "action": "BLOCK"  
},
```

```
{  
  
  "id": 5,  
  
  "name": "Block FTP",  
  
  "direction": "INBOUND",  
  
  "source": "ANY",
```



```
"destination": "LOCAL",  
  
"protocol": "TCP",  
  
"port": 21,  
  
"action": "BLOCK"  
  
},
```

```
# -----
```

```
# OUTBOUND RULES
```

```
# -----
```

```
{  
  
  "id": 6,  
  
  "name": "Allow DNS",  
  
  "direction": "OUTBOUND",  
  
  "source": "LOCAL",  
  
  "destination": "ANY",  
  
  "protocol": "UDP",  
  
  "port": 53,  
  
  "action": "ALLOW"  
  
},
```

```
{
```

```
"id": 7,  
  
"name": "Allow HTTPS Outbound",  
  
"direction": "OUTBOUND",  
  
"source": "LOCAL",  
  
"destination": "ANY",  
  
"protocol": "TCP",  
  
"port": 443,  
  
"action": "ALLOW"  
},
```

```
{  
  
"id": 8,  
  
"name": "Block Telnet Outbound",  
  
"direction": "OUTBOUND",  
  
"source": "LOCAL",  
  
"destination": "ANY",  
  
"protocol": "TCP",  
  
"port": 23,  
  
"action": "BLOCK"  
},
```

```
{
```

```
"id": 9,  
  
"name": "Allow DNS TCP",  
  
"direction": "OUTBOUND",  
  
"source": "LOCAL",  
  
"destination": "ANY",  
  
"protocol": "TCP",  
  
"port": 53,  
  
"action": "ALLOW"  
  
}  
  
]
```

```
# =====
```

```
# 4. FIREWALL LOG DATABASE
```

```
# =====
```

```
firewall_logs = []
```

```
# =====
```

```
# 5. DISPLAY FIREWALL STATUS
```

```
# =====
```

```
def display_firewall_status():
```

```
    print("\n")
```

```
    print("=" * 80)
```

```
    print("FIREWALL STATUS")
```

```
    print("=" * 80)
```

```
    if firewall_enabled:
```

```
        print("Firewall Status : ENABLED")
```

```
    else:
```

```
        print("Firewall Status : DISABLED")
```

```
    print("Default Policy :", default_policy)
```

```
    print("=" * 80)
```

```
# =====
```

```
# 6. DISPLAY FIREWALL RULES
```

```
# =====
```

```
def display_firewall_rules():
```

```
    print("\n")
```

```
    print("=" * 120)
```

```
    print("CONFIGURED FIREWALL RULES")
```

```
    print("=" * 120)
```

```
    print(
```

```
        f"{'ID':<5}"
```

```
        f"{'RULE NAME':<27}"
```

```
        f"{'DIRECTION':<12}"
```

```
        f"{'SOURCE':<18}"
```

```
        f"{'PROTOCOL':<10}"
```

```
        f"{'PORT':<8}"
```

```
        f"{'ACTION':<10}"
```

```
    )
```

```
    print("-" * 120)
```

```
    for rule in firewall_rules:
```

```
        print(
```

```
f"{rule['id']:<5}"  
  
f"{rule['name']:<27}"  
  
f"{rule['direction']:<12}"  
  
f"{rule['source']:<18}"  
  
f"{rule['protocol']:<10}"  
  
f"{rule['port']:<8}"  
  
f"{rule['action']:<10}"  
  
)
```

```
print("=" * 120)
```

```
# =====
```

```
# 7. IP ADDRESS MATCHING FUNCTION
```

```
# =====
```

```
def ip_matches(rule_address, packet_address):
```

```
    # ANY means every address is accepted
```

```
    if rule_address.upper() == "ANY":
```

```
        return True
```

```
# LOCAL represents the local computer
```

```
if rule_address.upper() == "LOCAL":
```

```
    return packet_address.upper() == "LOCAL"
```

```
# Exact IP comparison
```

```
return rule_address == packet_address
```

```
# =====
```

```
# 8. CHECK FIREWALL RULE
```

```
# =====
```

```
def check_firewall(
```

```
    direction,
```

```
    source,
```

```
    destination,
```

```
    protocol,
```

```
    port
```

```
):
```

```
    direction = direction.upper()
```

```
    protocol = protocol.upper()
```

```
# -----
```

```
# Firewall disabled
```

```
# -----
```

```
if not firewall_enabled:
```

```
    decision = "ALLOW"
```

```
    rule_name = "FIREWALL DISABLED"
```

```
    create_log(
```

```
        source,
```

```
        destination,
```

```
        protocol,
```

```
        port,
```

```
        direction,
```

```
        decision,
```

```
        rule_name
```

```
    )
```

```
    return decision, rule_name
```



```
# -----
```

```
# Check every firewall rule
```

```
# -----
```

```
for rule in firewall_rules:
```

```
    # Check direction
```

```
    if rule["direction"] != direction:
```

```
        continue
```

```
    # Check protocol
```

```
    if rule["protocol"] != protocol:
```

```
        continue
```

```
    # Check port
```

```
    if rule["port"] != port:
```

```
        continue
```

```
    # Check source IP
```

```
    if not ip_matches(rule["source"], source):
```

```
continue
```

```
# Check destination IP
```

```
if not ip_matches(rule["destination"], destination):
```

```
    continue
```

```
# Matching rule found
```

```
decision = rule["action"]
```

```
rule_name = rule["name"]
```

```
create_log(
```

```
    source,
```

```
    destination,
```

```
    protocol,
```

```
    port,
```

```
    direction,
```

```
    decision,
```

```
    rule_name
```

```
)
```

```
return decision, rule_name
```

```
# -----
```

```
# No rule found
```

```
# Apply default policy
```

```
# -----
```

```
decision = default_policy
```

```
rule_name = "DEFAULT DENY"
```

```
create_log(
```

```
    source,
```

```
    destination,
```

```
    protocol,
```

```
    port,
```

```
    direction,
```

```
    decision,
```

```
    rule_name
```

```
)
```

```
return decision, rule_name
```

```
# =====
```

```
# 9. CREATE FIREWALL LOG
```

```
# =====
```

```
def create_log(
```

```
    source,
```

```
    destination,
```

```
    protocol,
```

```
    port,
```

```
    direction,
```

```
    decision,
```

```
    rule_name
```

```
):
```

```
    log_entry = {
```

```
        "time":
```

```
            datetime.now().strftime("%H:%M:%S"),
```

```
        "source":
```

source,

"destination":

destination,

"protocol":

protocol,

"port":

port,

"direction":

direction,

"decision":

decision,

"rule":

rule_name

}

firewall_logs.append(log_entry)

```
# =====
```

```
# 10. TEST NETWORK TRAFFIC
```

```
# =====
```

```
def test_traffic(
```

```
    direction,
```

```
    source,
```

```
    destination,
```

```
    protocol,
```

```
    port
```

```
):
```

```
    print("\n")
```

```
    print("=" * 85)
```

```
    print("NETWORK TRAFFIC ANALYSIS")
```

```
    print("=" * 85)
```

```
    print("Direction :", direction)
```

```
    print("Source   :", source)
```

```
    print("Destination :", destination)
```

```
print("Protocol  :", protocol)
```

```
print("Port      :", port)
```

```
print("-" * 85)
```

```
decision, rule_name = check_firewall(
```

```
    direction,
```

```
    source,
```

```
    destination,
```

```
    protocol,
```

```
    port
```

```
)
```

```
print("Matched Rule:", rule_name)
```

```
print("Decision  :", decision)
```

```
if decision == "ALLOW":
```

```
    print("RESULT    : TRAFFIC ALLOWED")
```

```
else:
```

```
print("RESULT : TRAFFIC BLOCKED")
```

```
print("=" * 85)
```

```
# =====
```

```
# 11. ADD FIREWALL RULE
```

```
# =====
```

```
def add_firewall_rule(
```

```
    name,
```

```
    direction,
```

```
    source,
```

```
    destination,
```

```
    protocol,
```

```
    port,
```

```
    action
```

```
):
```

```
    new_id = 1
```

```
    if len(firewall_rules) > 0:
```



```
new_id = max(  
    rule["id"]  
    for rule in firewall_rules  
) + 1
```

```
new_rule = {  
  
    "id": new_id,  
  
    "name": name,  
  
    "direction":  
        direction.upper(),  
  
    "source":  
        source,  
  
    "destination":  
        destination,
```

```
"protocol":  
    protocol.upper(),  
  
"port":  
    port,  
  
"action":  
    action.upper()  
}
```

```
firewall_rules.append(new_rule)
```

```
print("\nFirewall rule added successfully.")
```

```
print(  
    "Rule ID   :",  
    new_rule["id"]  
)
```

```
print(  
    "Rule Name :",
```

```
new_rule["name"]
```

```
)
```

```
print(
```

```
    "Direction  :",
```

```
    new_rule["direction"]
```

```
)
```

```
print(
```

```
    "Protocol   :",
```

```
    new_rule["protocol"]
```

```
)
```

```
print(
```

```
    "Port       :",
```

```
    new_rule["port"]
```

```
)
```

```
print(
```

```
    "Action     :",
```

```
    new_rule["action"]
```

```
)
```

```
# =====
```

```
# 12. DELETE FIREWALL RULE
```

```
# =====
```

```
def delete_firewall_rule(rule_id):
```

```
    for rule in firewall_rules:
```

```
        if rule["id"] == rule_id:
```

```
            firewall_rules.remove(rule)
```

```
            print(
```

```
                "\nFirewall rule deleted:",
```

```
                rule["name"]
```

```
            )
```

```
        return
```

```
print("\nRule ID not found.")
```

```
# =====
```

```
# 13. ENABLE FIREWALL
```

```
# =====
```

```
def enable_firewall():
```

```
    global firewall_enabled
```

```
    firewall_enabled = True
```

```
    print("\nFirewall has been ENABLED.")
```

```
# =====
```

```
# 14. DISABLE FIREWALL
```

```
# =====
```

```
def disable_firewall():
```

```
global firewall_enabled
```

```
firewall_enabled = False
```

```
print("\nFirewall has been DISABLED.")
```

```
# =====
```

```
# 15. DISPLAY FIREWALL LOGS
```

```
# =====
```

```
def display_logs():
```

```
    print("\n")
```

```
    print("=" * 125)
```

```
    print("FIREWALL ACTIVITY LOG")
```

```
    print("=" * 125)
```

```
    if len(firewall_logs) == 0:
```

```
        print("No traffic has been processed.")
```

```
return
```

```
print(
```

```
    f"{'TIME':<10}"
```

```
    f"{'SOURCE':<18}"
```

```
    f"{'DESTINATION':<18}"
```

```
    f"{'PROTOCOL':<10}"
```

```
    f"{'PORT':<8}"
```

```
    f"{'DIRECTION':<12}"
```

```
    f"{'DECISION':<10}"
```

```
    f"{'RULE'}"
```

```
)
```

```
print("-" * 125)
```

```
for log in firewall_logs:
```

```
    print(
```

```
        f"{log['time']:<10}"
```

```
        f"{log['source']:<18}"
```

```
f"{log['destination']:<18}"  
  
f"{log['protocol']:<10}"  
  
f"{log['port']:<8}"  
  
f"{log['direction']:<12}"  
  
f"{log['decision']:<10}"  
  
f"{log['rule']}"  
  
)
```

```
print("=" * 125)
```

```
# =====
```

```
# 16. FIREWALL STATISTICS
```

```
# =====
```

```
def display_statistics():
```

```
    total = len(firewall_logs)
```

```
    allowed = 0
```



```
blocked = 0
```

```
for log in firewall_logs:
```

```
    if log["decision"] == "ALLOW":
```

```
        allowed += 1
```

```
    elif log["decision"] == "BLOCK":
```

```
        blocked += 1
```

```
print("\n")
```

```
print("=" * 70)
```

```
print("FIREWALL TRAFFIC STATISTICS")
```

```
print("=" * 70)
```

```
print("Total Traffic :", total)
```

```
print("Allowed      :", allowed)
```

```
print("Blocked   :", blocked)
```

```
if total > 0:
```

```
    allowed_percentage = (
```

```
        allowed / total
```

```
    ) * 100
```

```
    blocked_percentage = (
```

```
        blocked / total
```

```
    ) * 100
```

```
print(
```

```
    "Allowed %   :",
```

```
    round(allowed_percentage, 2),
```

```
    "%"
```

```
)
```

```
print(
```

```
    "Blocked %   :",
```

```
round(blocked_percentage, 2),  
  
"%"  
  
)
```

```
print("=" * 70)
```

```
# =====
```

```
# 17. MAIN PROGRAM
```

```
# =====
```

```
print("\n")
```

```
print("*" * 90)
```

```
print("    FIREWALL CONFIGURATION BASICS")
```

```
print("    INCOMING AND OUTGOING TRAFFIC CONTROL")
```

```
print("*" * 90)
```

```
# -----
```

```
# Display status
```

```
# -----
```

```
display_firewall_status()
```

```
# -----
```

```
# Display rules
```

```
# -----
```

```
display_firewall_rules()
```

```
# =====
```

```
# 18. INBOUND TRAFFIC TESTING
```

```
# =====
```

```
print("\n")
```

```
print("#" * 90)
```

```
print("      INBOUND TRAFFIC TESTING")
```

```
print("#" * 90)
```

```
# Test 1: HTTP
```

```
test_traffic(  
  
    direction="INBOUND",  
  
    source="192.168.1.50",  
  
    destination="LOCAL",  
  
    protocol="TCP",  
  
    port=80  
)
```

Test 2: HTTPS

```
test_traffic(  
  
    direction="INBOUND",  
  
    source="192.168.1.50",  
  
    destination="LOCAL",  
  
    protocol="TCP",  
  
    port=443  
)
```

Test 3: SSH from authorized administrator

```
test_traffic(  
  
    direction="INBOUND",
```

```
source="192.168.1.100",  
  
destination="LOCAL",  
  
protocol="TCP",  
  
port=22  
)
```

Test 4: SSH from unauthorized system

```
test_traffic(  
  
    direction="INBOUND",  
  
    source="192.168.1.200",  
  
    destination="LOCAL",  
  
    protocol="TCP",  
  
    port=22  
)
```

Test 5: Telnet

```
test_traffic(  
  
    direction="INBOUND",  
  
    source="192.168.1.50",  
  
    destination="LOCAL",
```

```
    protocol="TCP",  
  
    port=23  
)
```

Test 6: FTP

```
test_traffic(  
  
    direction="INBOUND",  
  
    source="192.168.1.50",  
  
    destination="LOCAL",  
  
    protocol="TCP",  
  
    port=21  
)
```

Test 7: Application Server

```
test_traffic(  
  
    direction="INBOUND",  
  
    source="192.168.1.50",  
  
    destination="LOCAL",  
  
    protocol="TCP",  
  
    port=8080  
)
```

```
)
```

```
# Test 8: Unknown Port
```

```
test_traffic(
```

```
    direction="INBOUND",
```

```
    source="192.168.1.50",
```

```
    destination="LOCAL",
```

```
    protocol="TCP",
```

```
    port=9999
```

```
)
```

```
# =====
```

```
# 19. OUTBOUND TRAFFIC TESTING
```

```
# =====
```

```
print("\n")
```

```
print("#" * 90)
```

```
print("          OUTBOUND TRAFFIC TESTING")
```

```
print("#" * 90)
```


Test 9: DNS UDP

```
test_traffic(  
  
    direction="OUTBOUND",  
  
    source="LOCAL",  
  
    destination="8.8.8.8",  
  
    protocol="UDP",  
  
    port=53  
  
)
```

Test 10: HTTPS

```
test_traffic(  
  
    direction="OUTBOUND",  
  
    source="LOCAL",  
  
    destination="google.com",  
  
    protocol="TCP",  
  
    port=443  
  
)
```

Test 11: Telnet

```
test_traffic(  
  
    direction="OUTBOUND",  
  
    source="LOCAL",  
  
    destination="192.168.1.50",  
  
    protocol="TCP",  
  
    port=23  
  
)
```

Test 12: DNS TCP

```
test_traffic(  
  
    direction="OUTBOUND",  
  
    source="LOCAL",  
  
    destination="8.8.8.8",  
  
    protocol="TCP",  
  
    port=53  
  
)
```

Test 13: Unknown outbound port

```
test_traffic(  
  
    direction="OUTBOUND",
```

```
source="LOCAL",

destination="192.168.1.50",

protocol="TCP",

port=9999

)

# =====

# 20. ADD NEW FIREWALL RULE

# =====

print("\n")

print("#" * 90)

print("          ADDING NEW FIREWALL RULE")

print("#" * 90)


add_firewall_rule(

    name="Allow Secure Application",

    direction="INBOUND",

    source="192.168.1.150",

    destination="LOCAL",
```

```
    protocol="TCP",  
  
    port=8443,  
  
    action="ALLOW"  
)
```

```
# =====
```

```
# 21. TEST NEW RULE
```

```
# =====
```

```
test_traffic(  
  
    direction="INBOUND",  
  
    source="192.168.1.150",  
  
    destination="LOCAL",  
  
    protocol="TCP",  
  
    port=8443  
)
```

```
# Unauthorized source
```

```
test_traffic(  
  
    direction="INBOUND",
```

```
source="192.168.1.200",  
  
destination="LOCAL",  
  
protocol="TCP",  
  
port=8443  
  
)
```

```
# =====
```

```
# 22. DISPLAY UPDATED RULES
```

```
# =====
```

```
display_firewall_rules()
```

```
# =====
```

```
# 23. DISPLAY LOGS
```

```
# =====
```

```
display_logs()
```

```
# =====
```

```
# 24. DISPLAY STATISTICS
```

```
# =====
```

```
display_statistics()
```

```
# =====
```

```
# 25. FINAL RESULT
```

```
# =====
```

```
print("\n")
```

```
print("*" * 90)
```

```
print("          PRACTICAL COMPLETED")
```

```
print("*" * 90)
```

```
print("""
```

```
Firewall simulation successfully demonstrated:
```

```
[1] Firewall status
```

```
[2] Firewall rules
```

```
[3] Inbound traffic
```

```
[4] Outbound traffic
```

- [5] TCP filtering
- [6] UDP filtering
- [7] Port-based filtering
- [8] Source IP filtering
- [9] Destination filtering
- [10] ALLOW action
- [11] BLOCK action
- [12] Default-deny policy
- [13] Authorized SSH access
- [14] Unauthorized SSH blocking
- [15] HTTP filtering
- [16] HTTPS filtering
- [17] DNS filtering
- [18] Telnet blocking
- [19] FTP blocking
- [20] Firewall logging
- [21] Firewall statistics
- [22] Adding firewall rules

RESULT:

Firewall rules were successfully simulated and network traffic was classified as ALLOW or BLOCK according to

the configured security policy.

```
""")
```

```
print("*" * 90)
```

19. Expected Output

When the code runs, you will see output similar to:

```
*****
```

```

FIREWALL CONFIGURATION BASICS
```

```

INCOMING AND OUTGOING TRAFFIC CONTROL
```

```
*****
```

```
=====
=====
```

```

FIREWALL STATUS
```

```
=====
=====
```

```

Firewall Status : ENABLED
```

```

Default Policy : BLOCK
```

```
=====
=====
```

The program then displays configured rules such as:

ID	RULE NAME	DIRECTION	SOURCE
1	Allow HTTP	INBOUND	ANY

- | | | | |
|---|-----------------------|----------|---------------|
| 2 | Allow HTTPS | INBOUND | ANY |
| 3 | Allow SSH from Admin | INBOUND | 192.168.1.100 |
| 4 | Block Telnet | INBOUND | ANY |
| 5 | Block FTP | INBOUND | ANY |
| 6 | Allow DNS | OUTBOUND | LOCAL |
| 7 | Allow HTTPS Outbound | OUTBOUND | LOCAL |
| 8 | Block Telnet Outbound | OUTBOUND | LOCAL |
-

20. Example Traffic Analysis

For HTTP:

Direction : INBOUND

Source : 192.168.1.50

Destination : LOCAL

Protocol : TCP

Port : 80

Matched Rule: Allow HTTP

Decision : ALLOW

RESULT : TRAFFIC ALLOWED

For Telnet:

Direction : INBOUND

Source : 192.168.1.50

Destination : LOCAL

Protocol : TCP

Port : 23

Matched Rule: Block Telnet

Decision : BLOCK

RESULT : TRAFFIC BLOCKED

For unauthorized SSH:

Source : 192.168.1.200

Protocol : TCP

Port : 22

Matched Rule: DEFAULT DENY

Decision : BLOCK

RESULT : TRAFFIC BLOCKED

This demonstrates why source IP filtering is useful.

21. Firewall Rule Logic Used in the Program

The program essentially performs the following logic:

IF firewall is disabled

THEN ALLOW

ELSE

Check direction

Check source IP

Check destination

Check protocol

Check port

IF matching rule exists

THEN perform rule action

ELSE

Apply DEFAULT DENY

In mathematical/pseudocode form:

IF Firewall = OFF

Decision = ALLOW

ELSE IF Matching_Rule = TRUE

Decision = Rule.Action

ELSE

Decision = BLOCK

22. Why This Program Is Suitable for Google Colab

This program uses only standard Python:

```
from datetime import datetime
```

It does not require:

pip install

external packages

administrator privileges

Windows PowerShell

Linux firewall utilities

network interfaces

Therefore, it can be executed directly in a Google Colab notebook.

23. Important Technical Clarification

This Python program is a firewall simulation, not an operating-system firewall.

For example:

```
test_traffic(
```

```
    direction="INBOUND",
```

```
    source="192.168.1.50",
```

```
destination="LOCAL",  
  
protocol="TCP",  
  
port=80  
  
)
```

does not actually open port 80 on your computer.

Instead, the Python program asks:

"According to my simulated firewall rules, should this traffic be allowed or blocked?"

The program then returns:

ALLOW

or:

BLOCK

Actual Firewall

An actual firewall operates at the operating-system or network-device level and can actually affect packets.

For example, on Windows, actual firewall configuration can be performed using PowerShell commands such as:

New-NetFirewallRule

Those commands should not be executed in Google Colab because Colab is not your Windows operating system.

24. Difference Between Simulation and Actual Firewall

Feature	Python Simulation	Actual Firewall
Rule creation	Yes	Yes
Rule evaluation	Yes	Yes

Traffic simulation	Yes	Real traffic
Actual packet filtering	No	Yes
Logging	Simulated	Actual
IP filtering	Simulated	Actual
Port filtering	Simulated	Actual
Google Colab compatible	Yes	No, not as host firewall
Windows firewall modification	No	Yes
Administrator access	No	Usually required for configuration

25. Result

Result: The firewall rule simulation was successfully implemented in Python. The program successfully classified incoming and outgoing network traffic based on direction, source IP address, destination, protocol, port number, and configured firewall action. Allow, block, default-deny, logging, rule creation, and traffic statistics were successfully demonstrated.

26. Conclusion

A firewall is an essential component of network security because it controls communication between systems based on defined security policies.

In this practical, a complete firewall rule-processing system was implemented using Python. The program demonstrated:

Firewall Rules



Traffic Inspection



Source/Destination Verification



Protocol Verification



Port Verification



Rule Matching



ALLOW / BLOCK



Logging



Statistics